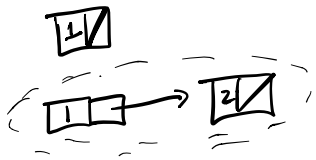
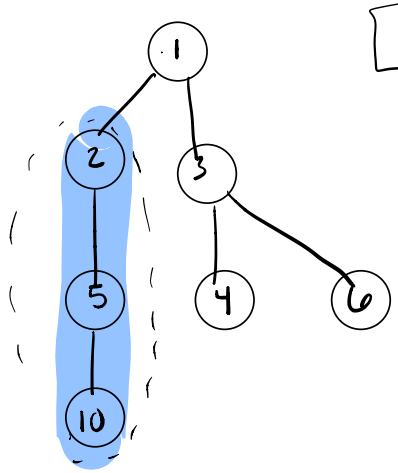


```
>>> a = Link(0)
>>> a.rest = a
```

a.rest.rest.rest.rest

```
def all_paths(t):
    if t.is_leaf():
        yield [t.label]
    else:
        for branch in t.branches:
            for path in all_paths(branch):
                yield [t.label] + path
```

Tree.is_leaf



[2, 5, 10]

[1, 2, 5, 10]

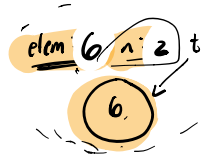
path + [t.label]

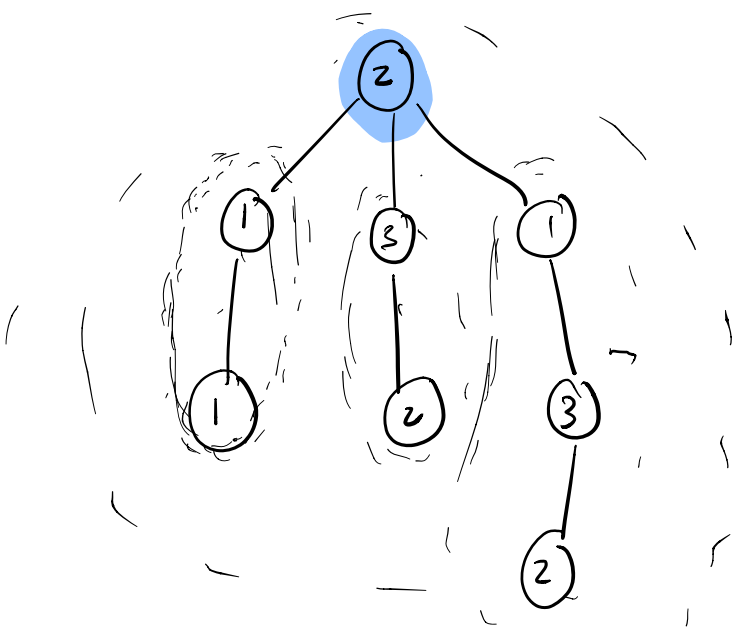
[2, 5, 10, 1]

[6]

```
def contains_n(elem, n, t):
    if n == 0:
        return True
    elif t.is_leaf():
        return t.label == elem and n == 1
    elif t.label == elem:
        return True in [contains_n(elem, n-1, b) for b in t.branches]
    else:
        return True in [contains_n(elem, n, b) for b in t.branches]
```

n > 0





at least n-1 instances

```
def subsets(lst,n):
    if n==0:
        return [[]]
    if lst==[]
        return [ ]
    case1 = [[lst[0]] + a for a in subsets(lst[1:],n-1)]
    case2 = subsets(lst[1:],n)
    return case1 + case2
```

[[]]

[[]]

lst n: 2

[1, 2, 3]

→ [[1,2] [1,3] [2,3]]

bad syntax lst[0] + subsets(lst[1:],n-1)

subsets(lst[1:],n)

lst n: 1

[1]

([ ], 1)

[[1]]

[[1]] + [ ]

[[1]]

[[1,2], [1,3], [2,3]]

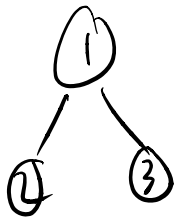
>>> subsets([2,3], 1)

[[2], [3]]

case1 = [[1,2], [1,3]]

case2 = [[2,3]]

[[1,2], [1,3], [2,3]]




```
def square_tree(t):
    return Tree(t.label**2, [square_tree(b) for b in
                                                t.branches])
```